

Communication Skill for Software and Knowledge Engineers

Lec. #03

Software Library Presentation

How to share team status updates

Why Team Status Updates Matter

Why Team Status Updates Matter

➤ **Team Alignment**

Keep everyone on the same page with progress, priorities, and expectations.

➤ **Early Risk Detection**

Surface blockers, delays, or dependencies before they become major issues.

➤ **Cross-Team Coordination**

Help other teams understand what you're doing and how it affects them.

➤ **Feedback Loops**

Create space for input, adjustments, and shared accountability.

➤ **Transparency & Trust**

Consistent, honest updates promote a culture of visibility and ownership.

Why Team Status Updates Matter

Without good updates, people assume the worst or nothing at all.

Myth	Reality
“Status updates are a waste of time”	Poorly done ones are — good ones prevent misalignment.
“Only managers care about updates”	Peer updates help teammates collaborate better.
“You have to write long reports”	The best updates are short, clear, and relevant.
“I’ll update when it’s done”	That’s too late — the point is to <i>show progress</i> .

Why It Matters to You

“Have you ever been frustrated by unclear or missing status updates?”

“What does a good status update help *you* do better?”

Purpose of Status Updates

 What Status Updates Are For

 What Status Updates Are NOT

What Status Updates Are For

1. Alignment and Transparency
2. Accountability and Progress Tracking
3. Risk Identification and Course Correction

What Status Updates Are For

1. Alignment and Transparency

- Everyone knows what's happening — no surprises.
- Keeps the team synchronized, especially in distributed/remote environments.
- Avoids duplicate work and confusion.

"Design and frontend know backend finished the API yesterday, so they can move forward today."

2.

3.

What Status Updates Are For

2. Accountability and Progress Tracking

- Helps people follow through on what they committed to.
- Creates a lightweight form of responsibility — not pressure, but clarity.
- Managers and team leads get visibility **without micromanaging**.

"Last week, we planned to complete login and registration.
This week, it's done and we're starting profile pages."

3.

What Status Updates Are For

3. Risk Identification and Course Correction

- Surfaces **blockers**, delays, or misalignments early.
- Enables the team to adjust plans or provide support **before it's too late**.

"We're waiting on the API schema, so frontend work is paused. Can we escalate or unblock this?"

What Status Updates Are For

1. Alignment and Transparency
2. Accountability and Progress Tracking
3. Risk Identification and Course Correction

Good updates *prevent fire-fighting* by revealing issues early — not hiding them.

What Status Updates Are NOT

1. Not Micromanagement
2. Not a Replacement for Real Problem-Solving
3. Not Just a Ritual for Ritual's Sake

What Status Updates Are NOT

1. Not Micromanagement

- The goal is not to spy or control people.
- Good updates create **autonomy with visibility**.
- When updates become overly detailed or forced, trust erodes.

2.

3.

What Status Updates Are NOT

2. Not a Replacement for Real Problem-Solving

- If a blocker is mentioned every day but never addressed the update is just noise.
- Status reports should **lead to action**, not replace it.

3.

What Status Updates Are NOT

3. Not Just a Ritual for Ritual's Sake

- If status updates feel like "checkbox exercises," people disengage.
- Updates must **deliver value** to the team: clarity, visibility, and forward motion.

Purpose of Status Updates

“If your project were a moving vehicle, status updates are the dashboard.

What happens when the dashboard is broken?”

Components of a Good Status Update

1. DONE: What Has Been Completed
2. IN PROGRESS: What You're Currently Working On
3. STUCK / NEEDS HELP: Where You're Blocked
4. NEXT STEPS: What's Coming Up
5. Optional: Metrics, Visuals, or Artifacts

Components of a Good Status Update

1. DONE: What Has Been Completed

Clearly state the work that's finished since the last update.

- Focus on **deliverables or outcomes**, not time spent.
- Use action-oriented language: “Completed,” “Merged,” “Deployed.”

Examples:

- ❖ “User authentication module has been integrated and tested.”
- ❖ “We finished the performance audit and removed 2 major bottlenecks.”

Avoid vague updates:

- ❖ “Worked on the login page” → *What was achieved? Is it finished?*

2.
3.
4.
5.

Components of a Good Status Update

2. IN PROGRESS: What You're Currently Working On

- Mention ongoing tasks and where they are in the workflow.
- Be brief, but specific enough to convey progress and intent.
- Flag any **blockers** or areas of uncertainty.

Examples:

- ❖ “Currently writing unit tests for the payments module.”
- ❖ “Working on UI for the new onboarding flow. Awaiting design spec.”

3.

4.

5.

Components of a Good Status Update

3. STUCK / NEEDS HELP: Where You're Blocked

This is the **most important section** — but often the most neglected.

- Be honest about blockers: it's not failure, it's part of collaboration.
- Ask for what you need: decision, feedback, access, clarification.
- Include **who** or **what** is needed to move forward.

Examples:

- ❖ “Blocked on backend team for updated API endpoint.”
- ❖ “Need final product decision on payment method options.”

4.

5.

Components of a Good Status Update

4. NEXT STEPS: What's Coming Up

Help others understand what's **next in the pipeline**.

- Set expectations clearly: what are you planning to do?
- Optionally include timing (e.g., “by end of week”).

Examples:

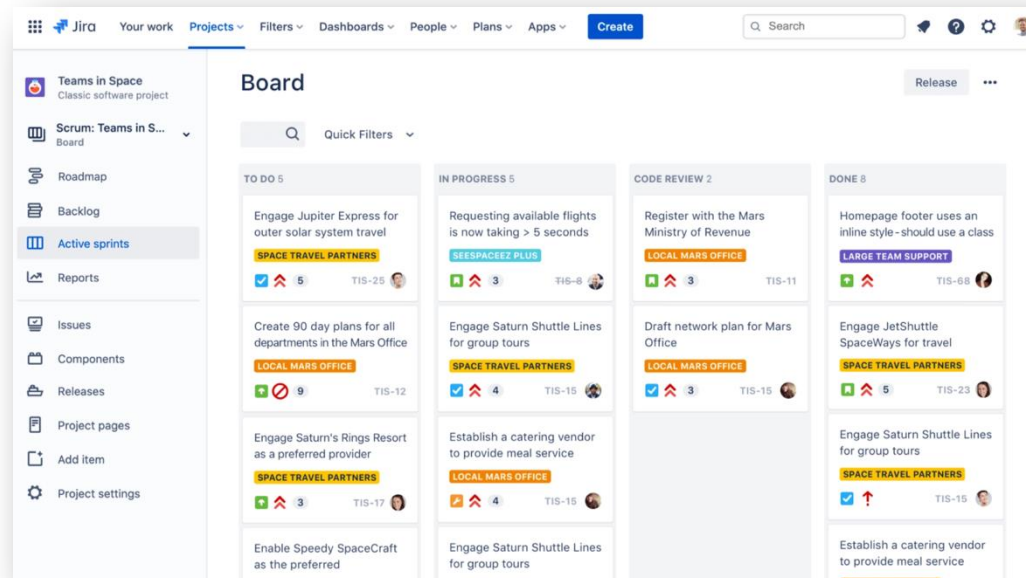
- ❖ “Tomorrow I’ll start working on the export-to-CSV feature.”
- ❖ “Next week we’ll begin integration testing with staging data.”

Components of a Good Status Update

5. Optional: Metrics, Visuals, or Artifacts

When relevant, share supporting information:

- Burndown charts, completion % (e.g., 8/10 stories done)
- Links to Jira tickets, dashboards, documents
- Screenshots or diagrams for visual updates



Tailoring for Your Audience

Not all updates are created equal — and not every stakeholder needs the same level of detail. The key is to **adjust the language, format, and focus** depending on **who's reading or listening**.

Different Audiences, Different Needs

Different Audiences, Different Needs



Team-Level Updates → Detail and Clarity



Manager/Executive Updates → Outcomes and Impact



Cross-Team Updates → Dependencies and Coordination

Different Audiences, Different Needs



Team-Level Updates → Detail and Clarity

- Focus on **task-level progress**, bugs, technical choices, and blockers.
- Provide specific information that team members need to move forward.
- Use direct references to tools, tickets, or code if needed.

“Switched from Axios to Fetch in the auth module to fix a cookie bug on Safari. Unit tests passed; integration testing in progress.”



Different Audiences, Different Needs



Manager/Executive Updates → Outcomes and Impact

- Summarize **what was achieved**, how it relates to goals, and **any risks**.
- Highlight **metrics, progress**, or decisions needed — not implementation detail.
- Keep it **high-level, concise, and time-bound**.

“Completed login redesign — improved sign-up conversion by 12%.
Integration with marketing tools is delayed; exploring fallback options.”



Different Audiences, Different Needs

Cross-Team Updates → Dependencies and Coordination

- Emphasize **interface points**, handoffs, and timeline overlaps.
- Use shared language — avoid internal jargon.
- Highlight **what others are waiting on, or what you need from them.**

“Frontend is ready for API v2 by Friday. Please confirm if backend delivery is still on track.”

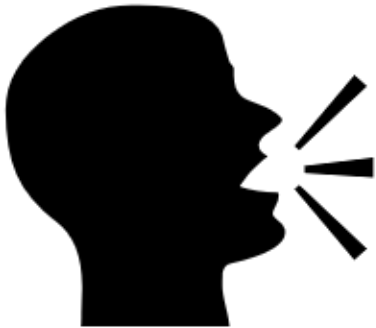
Tips for Audience-Tailored Updates

- **Use bullet points and clear structure**
Easy to skim and understand at a glance
- **Include dates/time frames**
“By Friday,” “this week,” “next sprint” avoids vagueness
- **Avoid unnecessary jargon**
Use plain language unless you’re speaking to technical peers
- **Be intentional with your focus**
Think: “What does this person need to know, decide, or unblock?”

Audience Grid

Audience	Needs to Know	Format	Language Style
Team	Tasks, blockers, next steps	Detailed	Technical
Manager	Progress, risks, impact	Summary	Plain + outcome-focused
Cross-Team	Dependencies, timelines	Shared goals	Neutral, cross-functional

Verbal vs Written Updates



Not all status updates are delivered the same way and **how** you communicate is just as important as **what** you communicate.



Verbal vs Written Updates

When to Use Each Format



Situation	Use Verbal Updates When...	Use Written Updates When...
Daily Syncs	Team needs real-time coordination	Team works across time zones
Sprint Planning / Demos	You're sharing live updates with discussion	You want a record others can revisit
Blockers or Escalations	A quick conversation can unblock faster	Details need to be documented
Project Milestones	Presenting to stakeholders or clients	Creating audit trails, links, reports



Verbal Updates

When speaking in meetings (standups, check-ins, calls):

- Speak with **confidence and focus** — avoid rambling.
- Use a **clear structure**: Done → Doing → Blocked → Next
- Keep it brief: **30–60 seconds** per person is ideal
- Avoid too much detail — speak to what's relevant **right now**

“Yesterday I finished the test suite for onboarding. Today I’m deploying to staging. Blocked on legal review for one text string.”



Write Updates

For async updates via Slack, Notion, email, or PM tools:

- Use **bullet points** or simple templates
- Add **headings or emojis** to break sections visually
- Link to supporting materials (JIRA ticket, dashboard, doc)
- Be concise, but **complete enough** that someone can act on it later



Frontend Update – June 5



Login flow UI complete



OAuth integration in progress (50%)



Blocked: Waiting on API spec (ETA: Friday)



Next: Begin dark mode styling



Progress: <https://linear.app/project/123>



- Standup Bots (for async updates)
 - ❖ Geekbot, DailyBot, Standuply (Slack/Teams integrations)
 - ❖ Automate daily check-ins with structure

- Project Management Tools
 - ❖ Notion, Confluence: Long-form, readable updates
 - ❖ JIRA, Linear, Trello: Task status, blockers, timelines
 - ❖ Slack/Teams: Fast, informal updates + integrations with tools

Best Practices for Effective Status Updates

1. Keep Updates Regular and Consistent
2. Share Both Wins and Blockers
3. Follow Through on Previous Updates
4. Clarify Assumptions and Deadlines

Best Practices for Effective Status Updates

1. Keep Updates Regular and Consistent

- Choose a **cadence**: daily, weekly, bi-weekly — and **stick to it**.
- Builds **trust and rhythm** in teams.
- Make updates part of the team's workflow, not an afterthought.
- Set reminders or use a standup bot for consistency.

2.
3.
4.

Best Practices for Effective Status Updates

2. Share Both Wins and Blockers

- Don't just say what's done — celebrate progress and expose risks.
- Blockers are **not failure** — surfacing them early = team support.

Example:

- ❖ “Wrapped up form validation.”
 - ❖ “Blocked on decision from marketing about dropdown options.”
- 3.
- 4.

Best Practices for Effective Status Updates

3. Follow Through on Previous Updates

- Reference past commitments to show **accountability**.
- Helps managers and peers understand momentum or delays.

Example:

- ❖ “Last week, I said I’d finish test coverage — it’s now at 90%, and I’m running edge case tests today.”

4.

Best Practices for Effective Status Updates

4. Clarify Assumptions and Deadlines

- Be explicit when dates or responsibilities are **tentative**.
- Don't assume people know what “soon” or “almost done” means.

Examples:

- ❖ “Expecting API update by Friday — backend team confirming.”
- ❖ “QA review starts tomorrow if we get UX sign-off today.”

Common Pitfalls

1. Being Too Vague
2. Oversharing Irrelevant Detail
3. Avoiding Problems or Blockers
4. Copy-Pasting Without Thinking

Common Pitfalls

1. Being Too Vague

“Still working on it.”

“Things are moving.”

 *Why it's bad:* No clarity, no action. Leaves others confused. 2.

 *Instead say:* 3.

“Finalizing form validation finishing by EOD if staging server is ready.” 4.

Common Pitfalls

2. Oversharing Irrelevant Detail

“Spent 2 hours debugging NPM dependency mismatch...”

 *Why it's bad:* Noise buries the signal. Not everything needs reporting.

 *Instead say:*

“Resolved environment issue with dependencies — dev setup is now stable.”

3.

4.

Common Pitfalls

3. Avoiding Problems or Blockers

Skipping blockers because you “hope to fix it soon.”

● *Why it's bad:* Delays help. Hidden problems compound silently.

✓ *Instead say:*

“Stuck on pagination bug — suspect issue with offset math. Might need backend help if unresolved by EOD.”

4.

Common Pitfalls

4. Copy-Pasting Without Thinking

Same status update every day → “Still working on login.”

 *Why it's bad:* Signals disengagement or lack of reflection.

 *Instead:*

Update phrasing or highlight what's changed:

“Login form mostly done; starting Google SSO integration today.”

Effective Status Updates

-  **Consistent**
-  **Clear**
-  **Outcome-focused**
-  **Honest**
-  **Relevant**

Daily Standups

- **Purpose:** Short, quick alignment on tasks and blockers
- **Format:** Typically verbal, 15 minutes max
- **Best for:** Agile teams, fast-paced environments

Questions to answer:

- ❖ What did I do yesterday?
 - ❖ What am I doing today?
 - ❖ Any blockers?
-
- Use a timer or speaking order to avoid derailment.

Weekly Check-ins

- **Purpose:** Reflect on broader progress and trends
- **Format:** Can be async or in-meeting
- **Best for:** Distributed teams, strategic reviews

Topics might include:

- ❖ Progress on key goals
 - ❖ Issues requiring decisions
 - ❖ Next week's priorities
-
- Include a “What’s one thing that went well?” moment to reinforce morale.

Asynchronous vs. Synchronous Updates

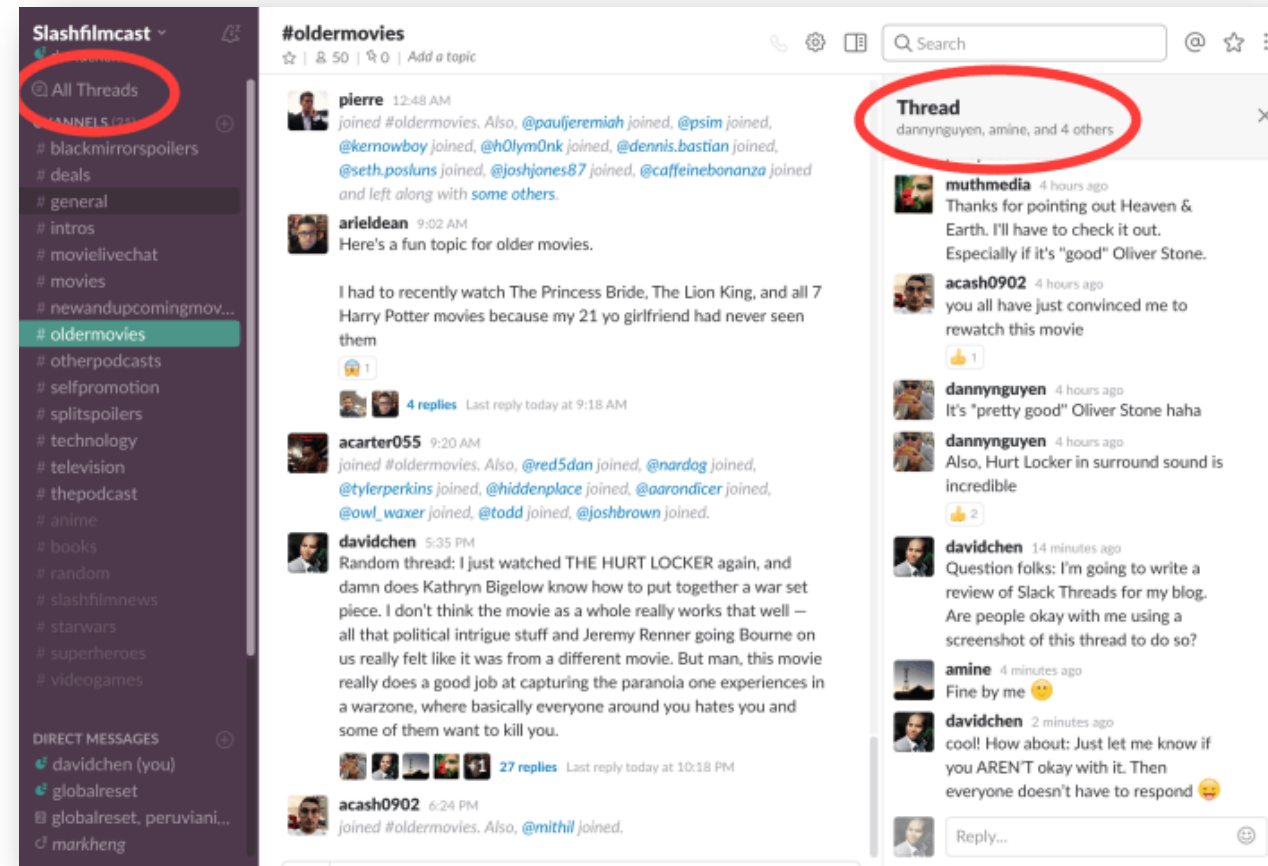
Style	Asynchronous	Synchronous
Pros	No time zones needed, searchable history	Immediate clarification, better for nuance
Cons	May get ignored, needs clear structure	Time-consuming, not inclusive for all
Tools	Slack bots, Notion, JIRA comments	Zoom, in-person meetings, team calls

Choose the style that fits your team's time zone and work rhythm.

Common Platforms

Slack Threads

- ✓ Good for lightweight, threaded updates
- ✓ Use emoji tags (✅, 🚧) and schedule reminders
- ✓ Integrates with tools like Jira, Linear, GitHub



Common Platforms

Google Docs / Sheets

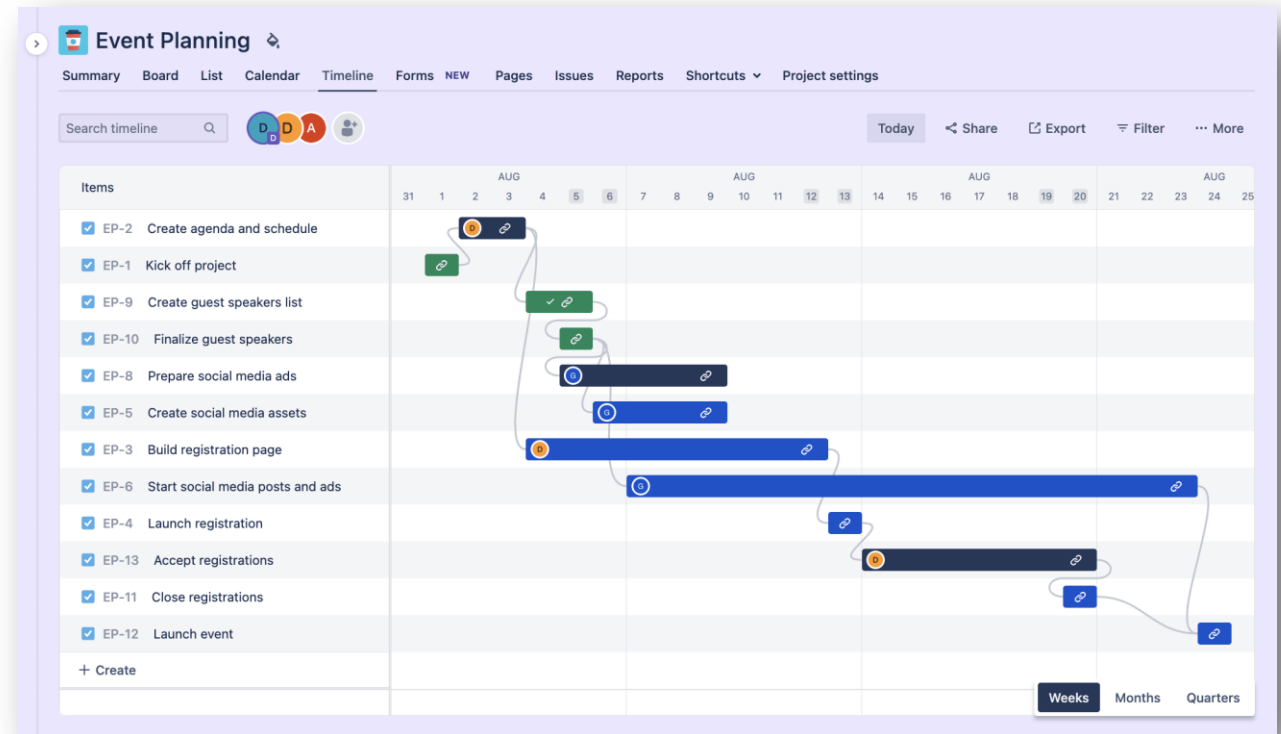
- Simple and familiar
- Great for structured weekly updates or logs
- Can include tables, checklists, and comments



Common Platforms

JIRA / Linear Boards

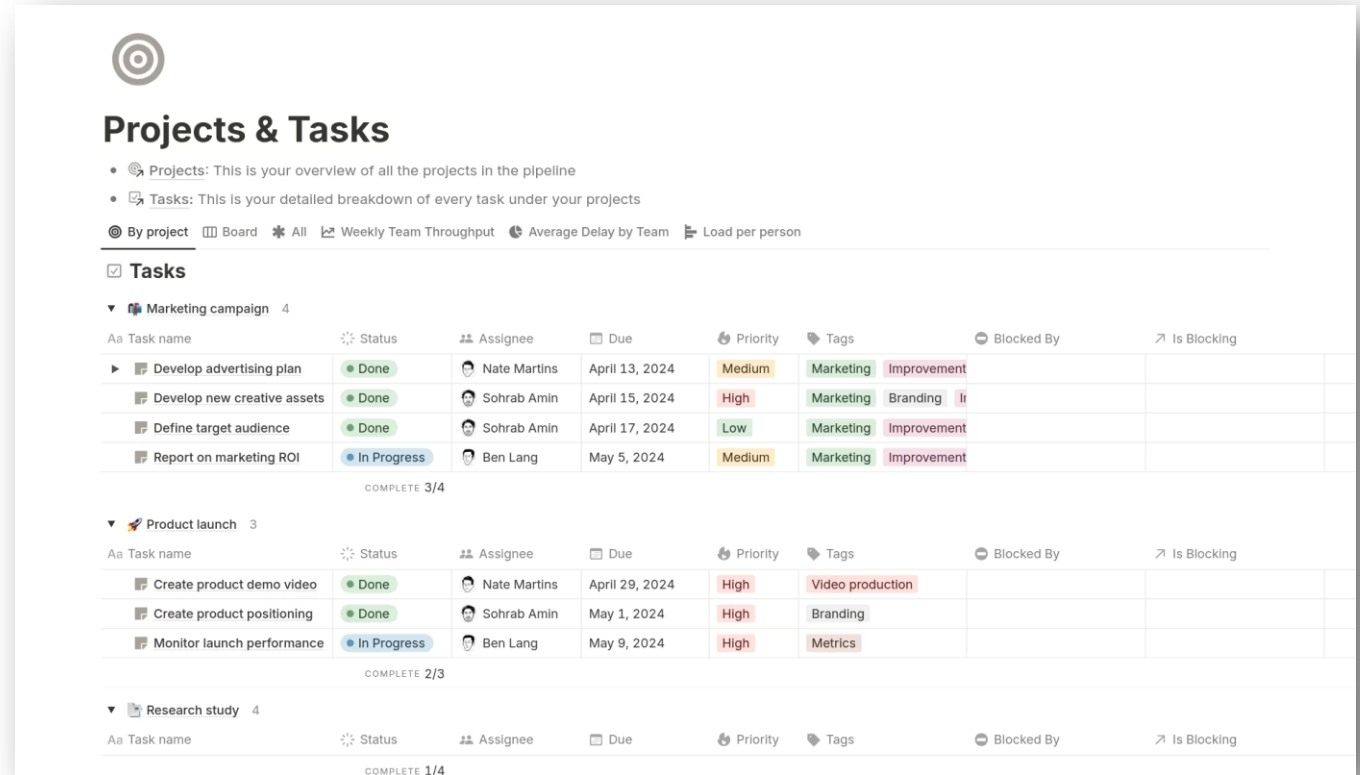
- Best for engineering product teams
- Use labels like "Blocked", "In Progress", "Done"
- Connect to sprint cycles and roadmaps



Common Platforms

Notion Templates

- Fully customizable
- Create shared update dashboards
- Combine notes, tasks, metrics, links in one place



Projects & Tasks

- **Projects:** This is your overview of all the projects in the pipeline
- **Tasks:** This is your detailed breakdown of every task under your projects

By project Board All Weekly Team Throughput Average Delay by Team Load per person

Tasks

▼ Marketing campaign 4

Task name	Status	Assignee	Due	Priority	Tags	Blocked By	Is Blocking
Develop advertising plan	Done	Nate Martins	April 13, 2024	Medium	Marketing Improvement		
Develop new creative assets	Done	Sohrab Amin	April 15, 2024	High	Marketing Branding		
Define target audience	Done	Sohrab Amin	April 17, 2024	Low	Marketing Improvement		
Report on marketing ROI	In Progress	Ben Lang	May 5, 2024	Medium	Marketing Improvement		

COMPLETE 3/4

▼ Product launch 3

Task name	Status	Assignee	Due	Priority	Tags	Blocked By	Is Blocking
Create product demo video	Done	Nate Martins	April 29, 2024	High	Video production		
Create product positioning	Done	Sohrab Amin	May 1, 2024	High	Branding		
Monitor launch performance	In Progress	Ben Lang	May 9, 2024	High	Metrics		

COMPLETE 2/3

▼ Research study 4

Task name	Status	Assignee	Due	Priority	Tags	Blocked By	Is Blocking
-----------	--------	----------	-----	----------	------	------------	-------------

COMPLETE 1/4

Example: Streamlit

The screenshot shows the GitHub interface for the `streamlit/streamlit` repository. The 'Pull requests' tab is selected, showing 124 open requests. A notification banner at the top provides guidance for first-time contributors. Below the filters, a list of pull requests is displayed, each with a title, status, labels, and a comment count.

First time contributing to streamlit/streamlit? [Dismiss](#) ...

If you know how to fix an [issue](#), consider opening a pull request for it.
You can read this repository's [contributing guidelines](#) to learn how to open a good pull request.

Filters Labels 185 Milestones 0 [New pull request](#)

124 Open	6,137 Closed	Author	Label	Projects	Milestones	Reviews	Assignee	Sort
[WIP] [AdvancedLayouts] Adds a width parameter to st.spinner ✓ change:feature impact:users security-assessment-completed 2								
#11547 opened 2 hours ago by sfc-gh-lwilby · Draft								
Bump the vitest group in /frontend with 2 updates ✗ change:chore impact:internal 1								
#11546 opened 2 hours ago by dependabot[bot] · Review required								
Bump the eslint group in /frontend with 2 updates ✗ change:chore impact:internal 1								
#11545 opened 2 hours ago by dependabot[bot] · Review required								
Bump ruff from 0.11.12 to 0.11.13 in /lib ✗ change:chore impact:internal 1								
#11543 opened 2 hours ago by dependabot[bot] · Review required								
[WIP] Add text & background color theming config options for st.info, st.error, st.success, st.warning ✗ change:feature impact:users security-assessment-completed 2								
#11542 opened 3 hours ago by mayagbarnes · Draft								
[WIP] [AdvancedLayouts] Adds a width parameter to st.radio • change:feature impact:users security-assessment-completed 2								
#11540 opened 3 hours ago by sfc-gh-lwilby · Draft								
Update test to only run on Chromium browser due to flakiness on WebKit and Firefox ✓ change:other impact:internal security-assessment-completed 2								
#11541 opened 3 hours ago by sfc-gh-lwilby · Draft								

Video

Q&A

Email: kundjanasith.th@ku.th